

Refactoring and Testing in 2016

A Practitioner's Perspective

Nat Pryce
nat@natpryce.com

ABSTRACT

We describe how and why we use test automation and refactoring in the design and evolution of software systems. The writing and running of automated tests against the system create mutual feedback loops that guide design decisions in both the system and test code.

CCS Concepts

•Software and its engineering → Designing software;
Software testing and debugging; Software evolution;
Acceptance testing; Collaboration in software development;
Agile software development;

Keywords

Design feedback, continual improvement, test automation, test-driven development

1. INTRODUCTION

As co-author of a book on test-driven development (TDD) [2], an invitation to talk for an hour to a captive audience of empirical software engineering researchers who study refactoring and testing was an opportunity I couldn't pass up. TDD has received some interest from the empirical research community, but many studies of TDD have not investigated outcomes that I, as a professional software developer, consider to be the primary reason for adopting TDD. They mostly focus on defect reduction [6], development of a single version of the software rather than sustainable long term product evolution [3] or development to a predefined specification without iterating on the design [4].

I took the opportunity to describe how and why my colleagues and I use test automation and refactoring to guide the design and sustainable evolution of software systems. I hope this inspires research into those practices and desired outcomes, as distinct from the use of test automation for defect detection.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

XP '16 Workshops May 24-24, 2016, Edinburgh, Scotland UK

© 2016 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-4134-9/16/05.

DOI: <http://dx.doi.org/10.1145/2962695.2962715>

2. INTERNAL AND EXTERNAL QUALITY

The interplay between external and internal quality is key to how we approach testing and refactoring in our development process.

External quality Does the system meet the current needs of its users and stakeholders?

Internal quality Can the system be easily modified to meet future needs?

This distinction between current needs and an uncertain future is common in the way software developers explain design. For example:

- Stevens, Myers and Constantine introduced the concept of coupling and cohesion in [5]. A module is cohesive if its elements belong together, which we can judge while reading the code. Modules are coupled if they must change at the same time, which we will only know for sure in the future when we have a reason to change them.
- XP distinguishes between acceptance and unit testing [1]. Acceptance testing validates present functionality. Unit testing requires the code be modular, so that it is easier to change in the future.

Uncertainty about the future is always on the software designer's mind.

My co-presenter at XP'16, Duncan McGregor, describes software development as making progress along an invisible line. On one side of the line, the team is working too quickly, spending too little time on design, refactoring, testing and automation, and the project will be overwhelmed by technical chaos. On the other they are working too slowly, spending too much time polishing the design rather than delivering immediate value, and the project will be overtaken in the market or by intra-organisational politics. The more experienced the engineer, the closer they can get to that line. They know when they need to cross from the predictable to chaotic side, when they've been on the chaotic side too long, and how to get back to the predictable side as fast as possible.

Test automation and refactoring are two of the tools we use to stay close to, and on the right side, of that invisible line.

3. DESIGN, TESTING AND FEEDBACK

3.1 Automating Tests

Figure 1 illustrates the common attitude to test automation: that automated tests merely give us feedback about the external quality of the system we're building.

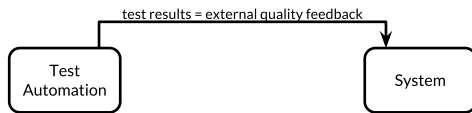


Figure 1: Feedback from running tests

However, our automated tests are not perfect – defects slip past our automated tests and are only caught by manual testing or result in failures in production. The test code itself has external quality that we must monitor and improve.

3.2 Testing the Test Code

Bug reports, monitoring and user feedback from our production system indicate where our tests are inadequate. We can also get useful information during development by instrumenting the system we are testing with code coverage tools and by “testing the tests” with mutation testing tools.

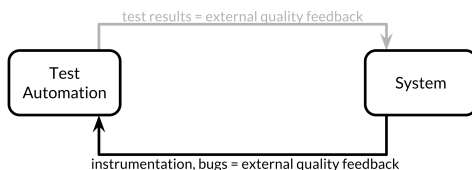


Figure 2: Feedback on external quality of the tests

Test automation on agile projects relies mostly on example based tests that continually test the same examples again and again, and see the same successful results. I have seen large products with many critical defects that have a passing test suite containing thousands of example based tests. I find it useful to thinking of testing in terms of search optimisation, rather than examples. The bugs are there; our tests should be looking for them!

We can improve our tests by randomising inputs, with property based testing and fuzz testing tools, for example. But what about repeatability? Actually, we don't need repeated tests, only repeatable test failures so that we can diagnose and fix them.

3.3 Test Automation Guides Design

A less common view of test automation, especially in academia, is that writing the tests give us feedback on the internal quality of our system. This is the basis of Test-Driven Development – using the difficulty in writing tests to guide the design of the system being tested.

In a test-driven approach, we ensure that it is always easy to write the next test. When it is difficult to write a test or get the test to run reliably, we refactor to improve the design of the system until the test test is easy to write. Then we write the test – there's no excuse now!

As a software designer, I do sometimes find it difficult to accept this feedback. It is tempting to blame the testing tools instead of my design choices, and add ever more

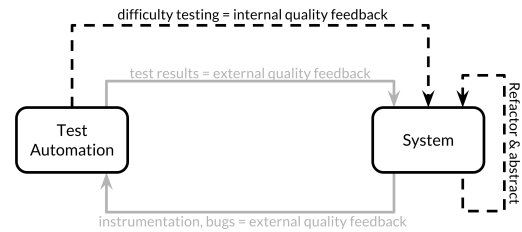


Figure 3: Feedback from writing automated tests

“clever” technical aids to the tests: reflection, byte-code transformation, preprocessor macros, tricks with the dynamic linker. But I have come to learn that although a test-driven approach to design may sometimes not be easy for the test I am currently working on, doing it continually makes development easy and more predictable in the long run¹.

It is difficult to create feedback from testing to system design when test automation is performed by a separate QA team. Software designers need to know what makes their system hard to test so that they can improve it's design, but do not often take into account the needs of Software Engineers in Test, or accept their feedback on design issues. In large projects, these communication channels need to be supported by management. If ease and reliability of testing is not considered an important feature of the system, the system will be difficult and expensive to test, and the automated test suite will become an unmaintainable legacy that hinders, rather than helps, the software's continued evolution.

We discovered an unexpected benefit when applying test-driven development feedback loops to system testing. Both test automation and system administration need similar features from the system. Changing the system to better support automated testing also make it easier to manage in production. To reliably test a system, we must be able to determine what the system is doing, detect when it has stopped doing it, detect when the system has failed, explain why has it failed, and restore the system to a known good state after failure. If our system tests are unreliable, that's a sign that we need to add interfaces to our system through which tests can better observe, synchronise with and control the activity of the system. Those changes turned out to be exactly what we need to better manage the systems we built. We end up using the same interfaces that the system exposed to the tests to build tools for automated and manual system management.

3.4 Guiding the Design of Test Automation

Good software design is vital in test automation code. The better the test code is designed, the easier it is to adapt it to changes in the system under test. Without good design, automated tests become an unmaintainable legacy. The same design principles that apply to the production code also apply to test code: Don't Repeat Yourself, low coupling, high cohesion, information hiding, type safety, etc. The same design practices that apply to the production code also apply to test code: continual refactoring to address duplication,

¹I have not seen this hunch supported or refuted by empirical research

improve names, and reduce risk when changing the test code.

The ease in which we can adapt the test suite to functional changes in the system provides feedback on the internal quality of our tests and highlights where the test code needs to be refactored.

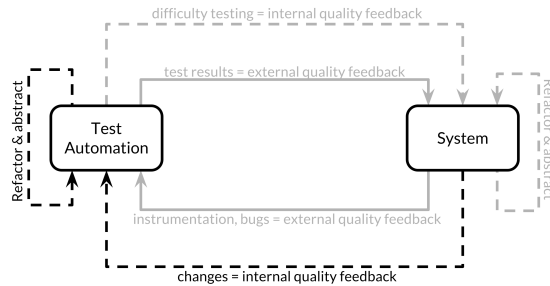


Figure 4: Feedback on internal quality of test code

When we follow XP engineering practices to write test code, the right abstractions for testing the application emerge from refactoring. A sign that the process is working well is that it becomes faster to write new tests as the project progresses, and the tests themselves require less code.

This takes more effort to achieve when testers and developers work separately. The less contact between development and test teams, the more the test teams will be surprised by changes in the builds delivered to them. The teams must maintain frequent communication so that test automation engineers are aware of upcoming changes to the system and do not build test abstractions that do not apply to future behaviour.

3.5 Eliminating Tests by Design

A desire for more predictable delivery with higher quality often translates into more test automation. More testing alone finds more bugs and makes delivery less predictable (albeit of eventual products with higher quality). To eliminate defects and improve predictability, we work to continually improve the design of the system.

- Testing detects defects
- Design avoids defects

When we detect a failure of the system, it's easy to conclude that the failure was the fault of a defect introduced by "developer error", fix the defect and move on. However, this does not help make development easier and more predictable. We must understand *why* developers introduced the defect. We can then change the design so that the error is rejected earlier in the process, or, ideally, never even occurs to developers because the design guides their thinking away from the erroneous constructs. Our aim is to eliminate not individual errors but entire classes of error.

These design improvements can be as simple as changing names to avoid misunderstandings or introducing value types into the domain model instead of using language primitives so that it is impossible to pass values to the wrong function parameters. But sometimes more extensive changes are worthwhile. For example, we have begun replacing Java code with Kotlin to remove the possibility of null pointer exceptions at runtime. Kotlin tracks nullability in the type system, and rejects null pointer dereferences at compile time.

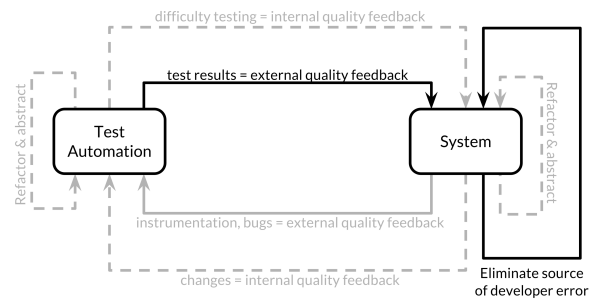


Figure 5: Eliminate entire classes of error and the tests that check for them

4. THE TEST AUTOMATION AND PRODUCTION CODE FORM A SYSTEM

The mutual feedback loops between test automation and production code means that it doesn't make much sense to think of having "a system" and "its tests". Both form a system. It's more useful to think in terms of components that are deployed in different configurations and environments, that act on one another and whose development affects each others' continued evolution.

This perspective allows one to consider of different configurations and approaches. For example:

- Why not test against the production deployment? What would we have to change to make that possible?
- Why not deploy tests into the production system? What useful feedback would that give us?
- How can we use monitoring to drive development as we do with tests²?

Answers to questions such as these introduce yet more feedback loops to our design and testing process.

5. REFERENCES

- [1] K. Beck. *Extreme Programming Explained: Embrace Change*. Number c. 1999.
- [2] S. Freeman and N. Pryce. *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley Professional, 1st edition, 2009.
- [3] N. Nagappan, E. M. Maximilien, T. Bhat, and L. Williams. Realizing quality improvement through test driven development: Results and experiences of four industrial teams. *Empirical Software Engineering*, 13(3):289–302, 2008.
- [4] J. Sanchez, L. Williams, and E. M. Maximilien. A longitudinal study of the use of a test-driven development practice in industry. In *Proc. Agile*, 2007.
- [5] W. P. Stevens, G. J. Myers, and L. L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974.
- [6] J. W. Wilkerson, J. F. Nunamaker, and R. Mercer. Comparing the defect reduction benefits of code inspection and test-driven development. *IEEE Transactions on Software Engineering*, 38(3):547–560, 2012.

²known variously as Monitoring-Driven or Metrics-Driven Development